MVP PRODUCT AND SYSTEM DESIGN SPEC

Fiton Room

A mobile virtual try-on app that lets shoppers upload full-body photos, submit clothing from online stores, and receive AI-generated try-on results.

Version

First product version with no user login and device-based history.

Primary Platform

React Native mobile app with iOS-specific share extension support.

Core Backend

Node.js API, Postgres, S3, SQS, worker processing, and GenAI try-on provider.

MVP Goal

Validate whether users find virtual try-on useful, fast enough, and visually convincing.

Prepared for:
Fiton Room MVP planning

Document date:
May 1, 2026

Contents

1. Executive Summary

Fiton Room is a mobile virtual try-on app. A user uploads one to three full-body photos, submits a clothing item from an online store, and receives an AI-generated image showing the garment on their body.

The first version should avoid login and reduce friction. Instead of accounts, the app will create a secure app-generated device install ID and use it to keep each device's upload history, job history, and result gallery.

MVP IDENTITY	IMAGE STORAGE
No login; secure device install ID	**Amazon S3 + CloudFront**
BACKEND	**DATABASE**
Node.js API + async worker	**Supabase Postgres for MVP**

> **Recommendation:** Start without Kubernetes/EKS. Use S3, SQS, Lambda or ECS/Fargate worker, Postgres, and a third-party try-on API. Add Redis and heavier infrastructure after real usage proves the need.

2. Product Vision

Fiton Room should make online clothing discovery feel more personal. The product is not just an image generator; it is a shopping companion that turns product pages, screenshots, and shared links into personal try-on previews.

Primary MVP Outcome

Validate that users can complete the flow from uploaded photo to useful try-on image with minimal friction and enough visual quality to make them return.

Initial Target User

Everyday online shoppers who want to preview clothes on themselves before buying.

3. Version 1 User Flow

1 Open app	2 Create device identity
User opens Fiton Room without creating an account.	App creates a secure install ID and registers the device with the backend.
3 Upload photos	**4 Submit clothing**
User uploads one to three full-body photos using S3 pre-signed URLs.	User pastes a product URL, uploads a screenshot, or uses the iOS share extension.

5 Create job

Backend creates a try-on job and sends it to an async queue.

6 Generate result

Worker extracts garment data and calls the virtual try-on API.

7 Save and notify

Result is saved to S3, metadata is saved in Postgres, and a push notification may be sent.

8 View history

User sees result history tied to this device install ID.

4. MVP Feature Scope

Must Have

- React Native mobile app with Fiton Room branding.
- No-login first launch using app-generated device install ID.
- Upload one to three full-body photos.
- Paste product URL.
- Upload clothing screenshot.
- Create try-on job and show status: pending, processing, completed, failed.
- Display generated try-on result.
- Device-based result history.
- Delete uploaded photos and generated results from the device history.
- S3 storage for raw uploads and generated results.
- Postgres metadata storage.

Should Have

- iOS Share Extension using Swift.
- Push notification when processing completes.
- CloudFront image delivery.
- Basic retry for failed worker jobs.
- Product URL extraction cache.
- Internal admin dashboard for job review and failure debugging.

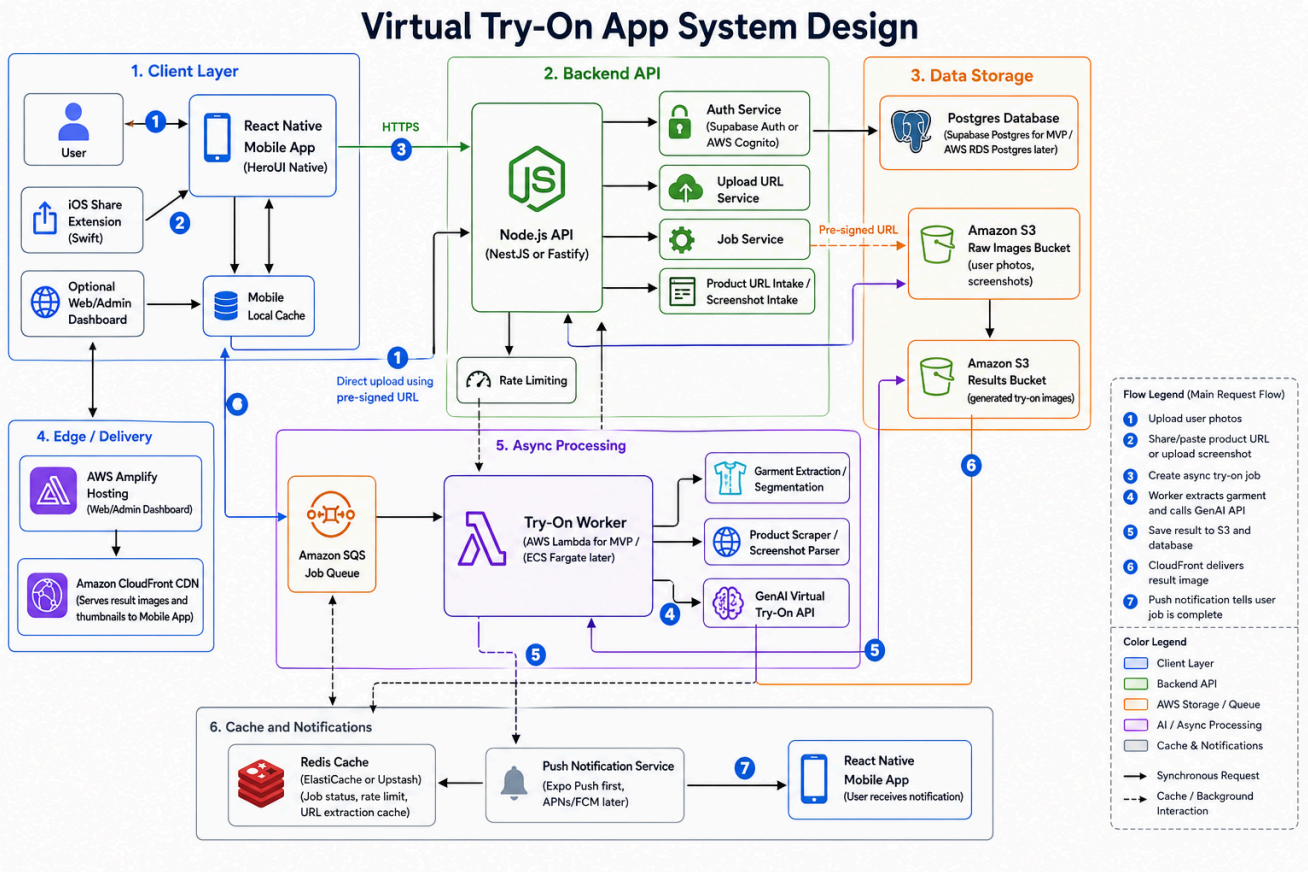
Later

- User accounts and cross-device sync.
- Android share support.
- Saved wardrobe.
- Multiple try-on variations per item.
- Paid credits or subscription plans.
- Affiliate links or ecommerce checkout integrations.
- In-house ML model.
- Kubernetes/EKS for mature scale.

5. Recommended Tech Stack

Layer	Choice	Reason
Mobile	React Native + TypeScript	Fast cross-platform mobile development.
UI	HeroUI Native	Native-focused UI layer matching the app plan.
iOS Extension	Swift / SwiftUI	Best fit for iOS share extension behavior.
Backend API	Node.js with NestJS or Fastify	Good developer speed, strong ecosystem, simple API work.
Database	Supabase Postgres	Fast MVP setup, free tier, real Postgres.
Object Storage	Amazon S3	Reliable storage for user uploads and generated images.
CDN	Amazon CloudFront	Faster delivery of thumbnails and results.
Queue	Amazon SQS	Simple async job orchestration.
Worker	Lambda first, ECS/Fargate later	Start simple; move to containers if jobs are long-running.
Cache	Redis later	Useful for rate limits, job status, and URL extraction cache after traffic grows.
Web/Admin Hosting	AWS Amplify Hosting	Good for a web dashboard, not for hosting the mobile app itself.

6. Initial System Design



Initial Fiton Room architecture: mobile app, backend API, S3, CloudFront, Postgres, SQS, worker processing, GenAI try-on API, cache, and push notifications.

```
React Native App
  -> Node.js Backend API
  -> Postgres Database
  -> S3 Raw Image Uploads
  -> SQS Try-On Job Queue
  -> Lambda/ECS Worker
  -> Garment Extraction + GenAI Try-On API
  -> S3 Result Images
  -> CloudFront CDN
  -> App Result Screen
  -> Push Notification
```

7. Service Connections

Mobile App to Backend API

The app calls the Node.js API over HTTPS. The API handles device registration, upload URL creation, job creation, job status, result history, deletion requests, and push token registration.

Mobile App to S3

The app uploads photos and screenshots directly to S3 using pre-signed URLs from the backend. Large image files should not pass through the API server.

Backend API to Postgres

Postgres stores device identities, photo metadata, product submissions, job records, results, and push tokens. The database should store S3 object keys, not raw images.

Backend API to SQS

When the app creates a try-on job, the backend stores a job row in Postgres and sends a compact message to SQS containing job ID and input references.

SQS to Worker

The worker pulls jobs from SQS, fetches source assets from S3, extracts garment/product details, calls the try-on API, writes the result image to S3, and updates Postgres.

S3 Results to CloudFront

Generated images and thumbnails are served through CloudFront for faster delivery and cleaner public access control.

Worker to Push Notifications

When a job completes or fails, the worker can trigger Expo Push for MVP. Direct APNs/FCM can replace this later if more control is needed.

8. Device-Based History Model

> **V1 decision:** Fiton Room will not require user login. Each app install will receive a backend-registered device install ID. History will be tied to that ID.

How It Works

1. On first launch, the app generates a random UUID.
2. The UUID is stored in secure local storage such as iOS Keychain or Expo SecureStore.
3. The app registers this ID with the backend as a device profile.
4. Photos, product inputs, jobs, and results are linked to the device profile.
5. The user can view history from the same device without logging in.

Important Notes

- Do not rely on hardware IDs, IDFA, serial numbers, or other sensitive tracking identifiers.
- If the user changes phone, history will not transfer until account login is added later.
- If the install ID is deleted, history may become unreachable from the app unless recovery is implemented.
- Add account migration later so a user can keep history across devices.

9. Initial Data Model

Table	Purpose	Key Fields
device_profiles	Anonymous v1 identity	id, install_id_hash, platform, app_version, created_at, last_seen_at
user_photos	Uploaded body photos	id, device_profile_id, s3_key, image_type, status, created_at
product_inputs	URL, screenshot, or share inputs	id, device_profile_id, input_type, source_url, screenshot_s3_key, extraction_status
try_on_jobs	Async generation jobs	id, device_profile_id, user_photo_id, product_input_id, status, failure_reason
try_on_results	Generated image records	id, job_id, device_profile_id, result_s3_key, thumbnail_s3_key, provider
push_tokens	Notification routing	id, device_profile_id, platform, token, enabled, updated_at
deletion_requests	Privacy and cleanup tracking	id, device_profile_id, status, requested_at, completed_at

10. Initial API Surface

Area	Endpoints
Device	POST /devices/register PATCH /devices/me GET /devices/me
Uploads	POST /uploads/photo-url POST /uploads/screenshot-url POST /uploads/complete
Product Inputs	POST /product-inputs/url POST /product-inputs/screenshot GET /product-inputs/:id
Try-On Jobs	POST /try-on-jobs GET /try-on-jobs/:id GET /try-on-jobs POST /try-on-jobs/:id/cancel
Results	GET /results GET /results/:id DELETE /results/:id
Push	POST /push-tokens DELETE /push-tokens/:id
Privacy	POST /privacy/delete-device-data

11. Caching Strategy

MVP Caching

- Mobile local cache for recent result thumbnails.
- CloudFront cache for result images and thumbnails.
- Database-backed job polling for the first version.

Add Redis When Needed

- Rate limiting by device install ID and IP.
- Fast job status updates.
- Duplicate product URL extraction cache.
- Temporary upload/session state.
- Worker progress updates.

12. Security and Privacy

- Use pre-signed S3 URLs with short expiration times.

- Keep S3 buckets private; serve approved results through CloudFront.
- Hash the app install ID before storing it in Postgres.
- Never use hardware identifiers as the primary user identity.
- Allow users to delete uploaded photos and generated results.
- Define raw photo retention before public launch.
- Show clear consent before uploading body photos.
- Log job failures without leaking sensitive image URLs.

> **Privacy risk:** Full-body photos are sensitive personal data. The MVP should include deletion, retention, and consent behavior from the start, even without user accounts.

13. Development Roadmap

Phase	Goal	Deliverables
Phase 1	Foundation	Mobile app shell, device registration, backend setup, Postgres schema, S3 upload flow.
Phase 2	Input and Job Flow	Photo upload, product URL input, screenshot upload, job creation, status screen.
Phase 3	AI Integration	Worker, product extraction, garment extraction, virtual try-on API, result saving.
Phase 4	History and Notifications	Device-based gallery, push notifications, deletion flow, error states.
Phase 5	Admin and Polish	Basic admin dashboard, job review, failure monitoring, launch hardening.

14. Open Questions

Product

1. Should the first release be iOS only, or iOS and Android?
2. Should the first clothing input be product URL, screenshot upload, or iOS share extension?
3. Should each try-on job generate one image or multiple variations?
4. Should raw photos expire automatically after a set number of days?
5. Should generated results be permanent until deleted?

Technical

1. Do we use Expo or bare React Native?
2. Do we choose NestJS or Fastify for the backend?
3. Which virtual try-on API provider should be tested first?
4. Should the MVP admin dashboard be included in the first build?
5. What is the target processing time for one try-on job?

Business

1. Will the MVP be free, credit-based, or subscription-based later?
2. How many free generations should each device receive?
3. Will Fiton Room support affiliate links or checkout integrations later?